CEE6501 — Lecture 13.1

Force Density Method (Linear Examples)

Learning Objectives

By the end of this lecture, you will be able to:

- explain

Agenda

- Part 1 -

Part 1 - Review of Force Density (Linear)

Define

$$\mathbf{D} = \mathbf{C}^T\mathbf{Q}\mathbf{C}, \qquad \mathbf{D}_f = \mathbf{C}^T\mathbf{Q}\mathbf{C}_f$$

Doing this for all 3 directions, the force density equations become

$$\mathbf{D}\mathbf{x} = \mathbf{p}_x - \mathbf{D}_f\mathbf{x}_f \tag{4-1}$$

$$\mathbf{D}\mathbf{y} = \mathbf{p}_y - \mathbf{D}_f\mathbf{y}_f \tag{4-2}$$

$$\mathbf{D}\mathbf{z} = \mathbf{p}_z - \mathbf{D}_f\mathbf{z}_f \tag{4-3}$$

These are **linear equations in the unknown coordinates**.

The Key Advantage: A Linear Formulation

For prescribed:

- connectivity
- support coordinates
- nodal loads
- force densities

the unknown free-node coordinates follow from a **single linear solve**.

That was the original appeal of the force density method.

In a planar case, once we solve

$$\mathbf{x} = \mathbf{D}^{-1}\left(\mathbf{p}_x - \mathbf{D}_f\mathbf{x}_f\right) \tag{5-1}$$

$$\mathbf{y} = \mathbf{D}^{-1}\left(\mathbf{p}_y - \mathbf{D}_f\mathbf{y}_f\right) \tag{5-2}$$

and for 3D we add in

$$\mathbf{z} = \mathbf{D}^{-1}\left(\mathbf{p}_z - \mathbf{D}_f\mathbf{z}_f\right) \tag{5-3}$$

Once $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$ are solved, the new equilibrium geometry is fully determined.

From there, we can:

1. reconstruct the updated nodal geometry
2. compute the corresponding branch lengths
3. recover the branch forces from

$$\mathbf{s} = \mathbf{Lq}$$

This is why the force density method is often described as a **one-step method**: once the force densities $\mathbf{q}$ are prescribed, the equilibrium geometry follows from a linear solve.

Solution Procedure

For a simple planar example with two free nodes, the workflow is:

1. choose the connectivity and build $\mathbf{C}$ and $\mathbf{C}_f$
2. prescribe the force densities and build $\mathbf{Q}$ (user chooses)
3. compute

$$\mathbf{D} = \mathbf{C}^T\mathbf{Q}\mathbf{C}$$

and

$$\mathbf{D}_f = \mathbf{C}^T\mathbf{Q}\mathbf{C}_f$$

4. solve separately for the unknown x- and y-coordinates
5. recover branch lengths and then branch forces

Part 2 - Code Implementation

Repository Link

https://github.com/Bruun-Automation-Research-Lab/matrix-form-finding

Running Code

Run from the `./formfinder/main.py` file

Text data saved to `./debug` folder

Output plots and daya saved to Text data saved to `./output` folder

Running from the `main.py` File

To run the linear force density solver, open `main.py` and set:

- `solver=solvers[1]` for `FD_linear`
- `structure=structs[...]` to whichever example you want to run

```python
if __name__ == "__main__":
    solvers = {
        1: "FD_linear",
        2: "FD_iter",
        3: "SM",
        4: "DR_imp",
        5: "DR_leap",
    }
    structs = {
        1: "Schek_2D",
        2: "Schek_Modified",
        3: "Veenendaal",
        4: "Pauletti",
        5: "CEE6501_2D",
        6: "CEE6501_PointLoad",
        7: "Star",
    }
    run = FormFinder(
        solver=solvers[1], structure=structs[1], debug=True, plot_save=True
    )
    run.solve()
```

Input File Structure

Input files specified in `./formfinder/structures` folder

Define as a function you can import

```python
def generate_struct():
    # ------------------------------------------------------------------------
    # Node coordinates
    # Outer support nodes (6-9) are fixed.
    # Inner nodes (1-5) define the central network.
    # All nodes lie in the XY plane, so z = 0 for every node.
    # ------------------------------------------------------------------------
    nodes = {
        1: (-2.0, 0.0, 0.0),
        2: (0.0, -2.0, 0.0),
        3: (2.0, 0.0, 0.0),
        4: (0.0, 2.0, 0.0),
        5: (0.0, 0.0, 0.0),
        6: (-3.0, 0.0, 0.0),
        7: (0.0, -3.0, 0.0),
        8: (3.0, 0.0, 0.0),
        9: (0.0, 3.0, 0.0),
    }

    # ------------------------------------------------------------------------
    # Element connectivity
    # Format: {element_id: (start_node, end_node)}
    # ------------------------------------------------------------------------
    elements = {
        1: (1, 6),
        2: (2, 7),
        3: (3, 8),
        4: (4, 9),
        5: (1, 4),
        6: (1, 2),
        7: (2, 3),
        8: (3, 4),
        9: (4, 5),
        10: (1, 5),
        11: (2, 5),
        12: (3, 5),
    }

```

```python
    # ------------------------------------------------------------------------
    # Initial element preload / force-density values
    # In FD_linear, this is the prescribed q value for each element.
    # ------------------------------------------------------------------------
    elements_preload = {
        1: 1.0,
        2: 1.0,
        3: 1.0,
        4: 1.0,
        5: 1.0,
        6: 1.0,
        7: 1.0,
        8: 1.0,
        9: 1.0,
        10: 1.0,
        11: 1.0,
        12: 1.0,
    }

    # ------------------------------------------------------------------------
    # Applied nodal loads
    # Format: {node_id: (Fx, Fy, Fz)}
    # No external loads are applied in this example.
    # ------------------------------------------------------------------------
    nodes_loads = {
        1: (0.0, 0.0, 0.0),
        2: (0.0, 0.0, 0.0),
        3: (0.0, 0.0, 0.0),
        4: (0.0, 0.0, 0.0),
        5: (0.0, 0.0, 0.0),
        6: (0.0, 0.0, 0.0),
        7: (0.0, 0.0, 0.0),
        8: (0.0, 0.0, 0.0),
        9: (0.0, 0.0, 0.0),
    }

    # ------------------------------------------------------------------------
    # Boundary conditions
    # 0 = free
    # 1 = fixed
    # ------------------------------------------------------------------------
    nodes_fixed = {
        1: 0,
        2: 0,
        3: 0,
        4: 0,
        5: 0,
        6: 1,
        7: 1,
        8: 1,
        9: 1,
    }

    return nodes, elements, elements_preload, nodes_loads, nodes_fixed

```

Why Define as Function?

Allows you to create more complex definition, rather than hard-coding all information

For example, defining nodal coordinates in the `CEE6501_PointLoad.py` structure

```
def generate_struct(rows=8, cols=10, spacing=1.0):
    # ------------------------------------------------------------------
    # Initialize dictionaries
    # ------------------------------------------------------------------
    nodes = {}
    # ------------------------------------------------------------------
    # Node coordinates
    # rows x cols rectangular grid in the XY plane with uniform spacing.
    # All nodes start with z = 0.0.
    # Node numbering proceeds row by row:
    # left to right, then bottom to top.
    # ------------------------------------------------------------------
    node_id = 1
    for i in range(rows):
        for j in range(cols):
            nodes[node_id] = (j * spacing, i * spacing, 0.0)
            node_id += 1
```

Run Flags

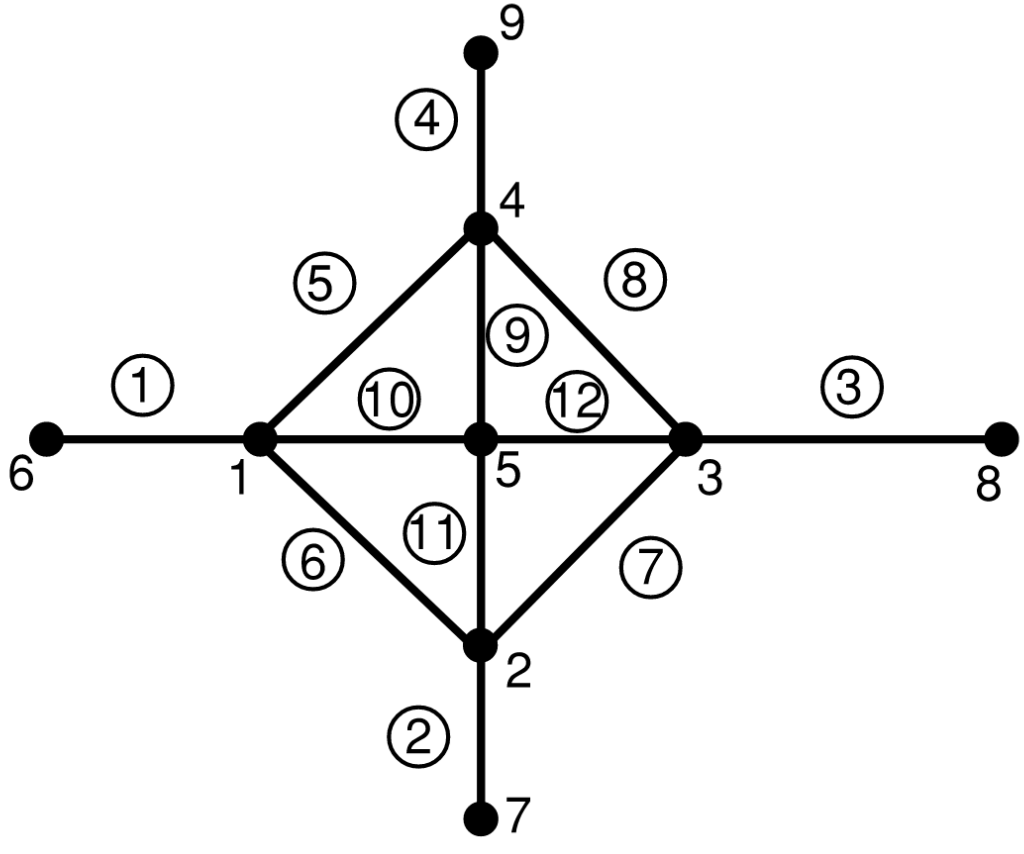
```
run = FormFinder(
    solver=solvers[1], structure=structs[1], debug=True, plot_save=True
)
```

- `debug=True`
  writes a detailed text log of key variables and results throughout the analysis and iterations to the `./debug` folder.
  Useful for understanding solver behavior, but can slow the run significantly for larger structures.

- `plot_save=True`
  saves the generated plots to the `./output` folder.s

Part 3 - Simple 2D Example (Schek Paper)

Description of Structure



Geometry Setup (Output in Debug)

NODES
[n x 3]

	x	y	z
	1 x	1 x	1 x
1	-2.00	0.00	0.00
2	0.00	-2.00	0.00
3	2.00	0.00	0.00
4	0.00	2.00	0.00
5	0.00	0.00	0.00
6	-3.00	0.00	0.00
7	0.00	-3.00	0.00
8	3.00	0.00	0.00
9	0.00	3.00	0.00

ELEMENTS
[m x 2] & [m x 1]

	n_i	n_j
	1 x	1 x
1	1	6
2	2	7
3	3	8
4	4	9
5	1	4
6	1	2
7	2	3
8	3	4
9	4	5
10	1	5
11	2	5
12	3	5

NODAL LOADS
[n x 3]

	p_x	p_y	p_z
	1 x	1 x	1 x
1	0.00	0.00	0.00
2	0.00	0.00	0.00
3	0.00	0.00	0.00
4	0.00	0.00	0.00
5	0.00	0.00	0.00
6	0.00	0.00	0.00
7	0.00	0.00	0.00
8	0.00	0.00	0.00
9	0.00	0.00	0.00

NODAL FIXITY
[n x 1]
1 = fixed | 0 = free

	fix?
	1 x
1	0
2	0
3	0
4	0
5	0
6	1
7	1
8	1
9	1

Geometry Setup (Branch-Node Matrices)

BRANCH-NODE MATRIX
[m x n]
C_total =

	1	2	3	4	5	6	7	8	9
1	1	.	.	.	.	-1	.	.	.
2	.	1	.	.	.	.	-1	.	.
3	.	.	1	.	.	.	.	-1	.
4	.	.	.	1	.	.	.	.	-1
5	1	.	.	-1	.	.	.	.	.
6	1	-1	.	.	.	.	.	.	.
7	.	1	-1	.	.	.	.	.	.
8	.	.	1	-1	.	.	.	.	.
9	.	.	.	1	-1	.	.	.	.
10	1	.	.	.	-1	.	.	.	.
11	.	1	.	.	-1	.	.	.	.
12	.	.	1	.	-1	.	.	.	.

BRANCH-NODE MATRIX (FREE)
[m x n_free]
C =

	1	2	3	4	5
1	1	.	.	.	.
2	.	1	.	.	.
3	.	.	1	.	.
4	.	.	.	1	.
5	1	.	.	-1	.
6	1	-1	.	.	.
7	.	1	-1	.	.
8	.	.	1	-1	.
9	.	.	.	1	-1
10	1	.	.	.	-1
11	.	1	.	.	-1
12	.	.	1	.	-1

BRANCH-NODE MATRIX (FIXEI
[m x n_fixed]
C_f =

	1	2	3	4
	6	7	8	9
1	-1	.	.	.
2	.	-1	.	.
3	.	.	-1	.
4	.	.	.	-1
5	.	.	.	.
6	.	.	.	.
7	.	.	.	.
8	.	.	.	.
9	.	.	.	.
10	.	.	.	.
11	.	.	.	.
12	.	.	.	.

Option 1 — Uniform Force Density

Assume the same force density for every member: $q_i = 1$

So the force density vector becomes

$$\mathbf{q} = [1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1]^T$$

Using this, we form the matrices

$$\mathbf{D} = \mathbf{C}^T\mathbf{Q}\mathbf{C} \qquad \mathbf{D}_f = \mathbf{C}^T\mathbf{Q}\mathbf{C}_f$$

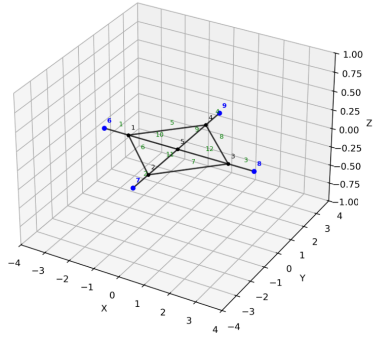
D (FREE)
[n_free x n_free]
D =
1 x

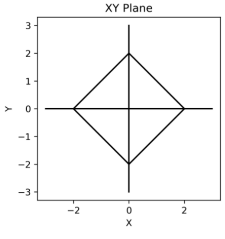
			1	2	3	4	5
			1	2	3	4	5
1	1		4.00	-1.00	.	-1.00	-1.00
2	2		-1.00	4.00	-1.00	.	-1.00
3	3		.	-1.00	4.00	-1.00	-1.00
4	4		-1.00	.	-1.00	4.00	-1.00
5	5		-1.00	-1.00	-1.00	-1.00	4.00

D (FIXED)
[n_free x n_fixed]
D_f =
1 x

			1	2	3	4
			6	7	8	9
1	1		-1.00	.	.	.
2	2		.	-1.00	.	.
3	3		.	.	-1.00	.
4	4		.	.	.	-1.00
5	5		.	.	.	.

Results (Start -> Final)



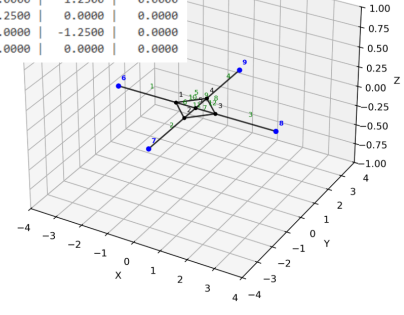


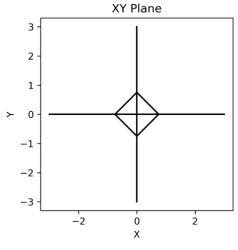
NODES
[n x 3]

	x	y	z
	1 x	1 x	1 x
1	-2.00	0.00	0.00
2	0.00	-2.00	0.00
3	2.00	0.00	0.00
4	0.00	2.00	0.00
5	0.00	0.00	0.00
6	-3.00	0.00	0.00
7	0.00	-3.00	0.00
8	3.00	0.00	0.00
9	0.00	3.00	0.00

DELTA DISPLACEMENTS (FREE)
[n_free x 1]

	Node	d_x	d_y	d_z
		1 x	1 x	1 x
1	1	1.2500	0.0000	0.0000
2	2	0.0000	1.2500	0.0000
3	3	-1.2500	0.0000	0.0000
4	4	0.0000	-1.2500	0.0000
5	5	0.0000	0.0000	0.0000





NODES (NEW)
[n x 3]

	x	y	z
	1 x	1 x	1 x
1	-0.75	0.00	0.00
2	0.00	-0.75	0.00
3	0.75	0.00	0.00
4	0.00	0.75	0.00
5	0.00	0.00	0.00
6	-3.00	0.00	0.00
7	0.00	-3.00	0.00
8	3.00	0.00	0.00
9	0.00	3.00	0.00

Final Node Positions and Branch Forces & Lengths

FINAL NODES

[n x 3]

	x	y	z
	1 x	1 x	1 x
1	-0.750	-0.000	0.000
2	-0.000	-0.750	0.000
3	0.750	-0.000	0.000
4	-0.000	0.750	0.000
5	-0.000	-0.000	0.000
6	-3.000	0.000	0.000
7	0.000	-3.000	0.000
8	3.000	0.000	0.000
9	0.000	3.000	0.000

FINAL FORCE/LENGTH/FORCE DENSITY

[m x 1]

	Force	Length	q
	1 x	1 x	1 x
1	2.2500	2.2500	1.0000
2	2.2500	2.2500	1.0000
3	2.2500	2.2500	1.0000
4	2.2500	2.2500	1.0000
5	1.0607	1.0607	1.0000
6	1.0607	1.0607	1.0000
7	1.0607	1.0607	1.0000
8	1.0607	1.0607	1.0000
9	0.7500	0.7500	1.0000
10	0.7500	0.7500	1.0000
11	0.7500	0.7500	1.0000
12	0.7500	0.7500	1.0000

Option 2 — Scaled Force Density

Scale all force densities for all members uniformly: $q_i = 5$

So the force density vector becomes

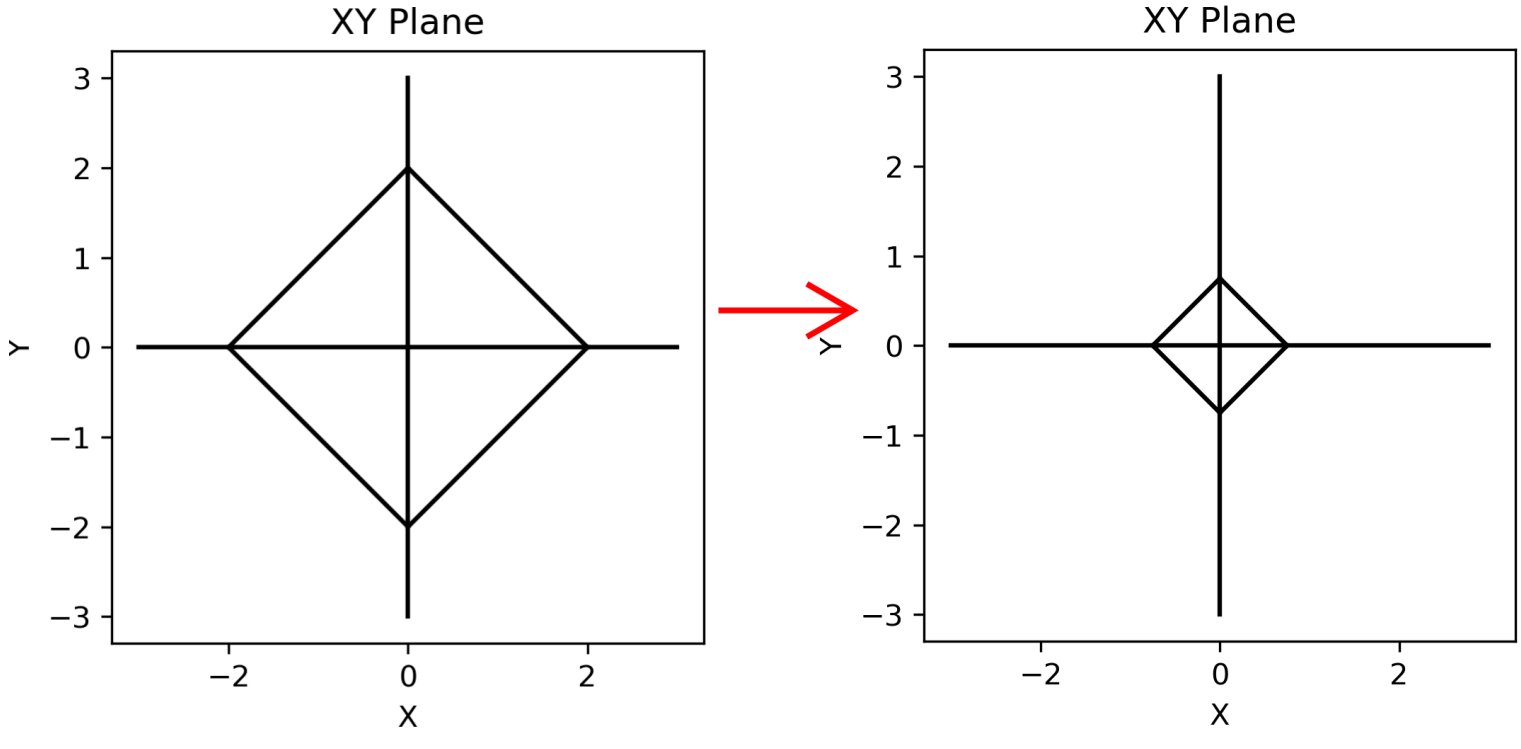
$$\mathbf{q} = [5,\ 5,\ 5,\ 5,\ 5,\ 5,\ 5,\ 5,\ 5,\ 5,\ 5,\ 5]^T$$

This gives the same *form* as Option 1.

Key idea:
Only the *relative* values of q matter — not their absolute magnitude.

Scaling all q by a constant changes the force level, but not the geometry.

Results (Start -> Final)



Final Node Positions and Branch Forces & Lengths

Same as in the $q = 1$ case!

FINAL NODES

[n x 3]

	x	y	z
	1 x	1 x	1 x
1	-0.750	-0.000	0.000
2	-0.000	-0.750	0.000
3	0.750	-0.000	0.000
4	-0.000	0.750	0.000
5	-0.000	-0.000	0.000
6	-3.000	0.000	0.000
7	0.000	-3.000	0.000
8	3.000	0.000	0.000
9	0.000	3.000	0.000

FINAL FORCE/LENGTH/FORCE DENSITY

[m x 1]

	Force	Length	q
	1 x	1 x	1 x
1	11.2500	2.2500	5.0000
2	11.2500	2.2500	5.0000
3	11.2500	2.2500	5.0000
4	11.2500	2.2500	5.0000
5	5.3033	1.0607	5.0000
6	5.3033	1.0607	5.0000
7	5.3033	1.0607	5.0000
8	5.3033	1.0607	5.0000
9	3.7500	0.7500	5.0000
10	3.7500	0.7500	5.0000
11	3.7500	0.7500	5.0000
12	3.7500	0.7500	5.0000

Option 3 — Non-Uniform Force Density

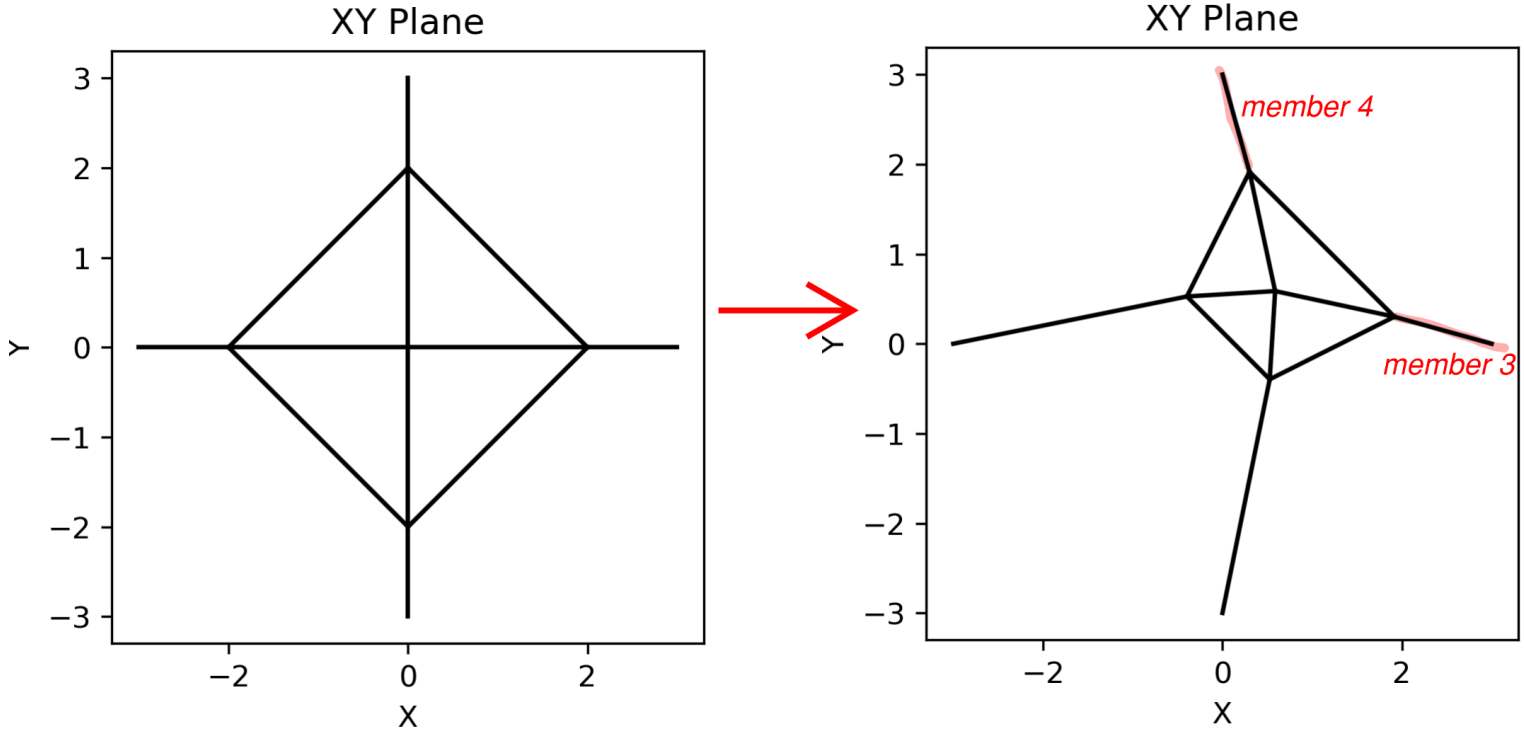
Assign different force density pattern to selected members:

$$q_i = \begin{cases} 4 & \text{for members 3 and 4} \\ 1 & \text{otherwise} \end{cases}$$

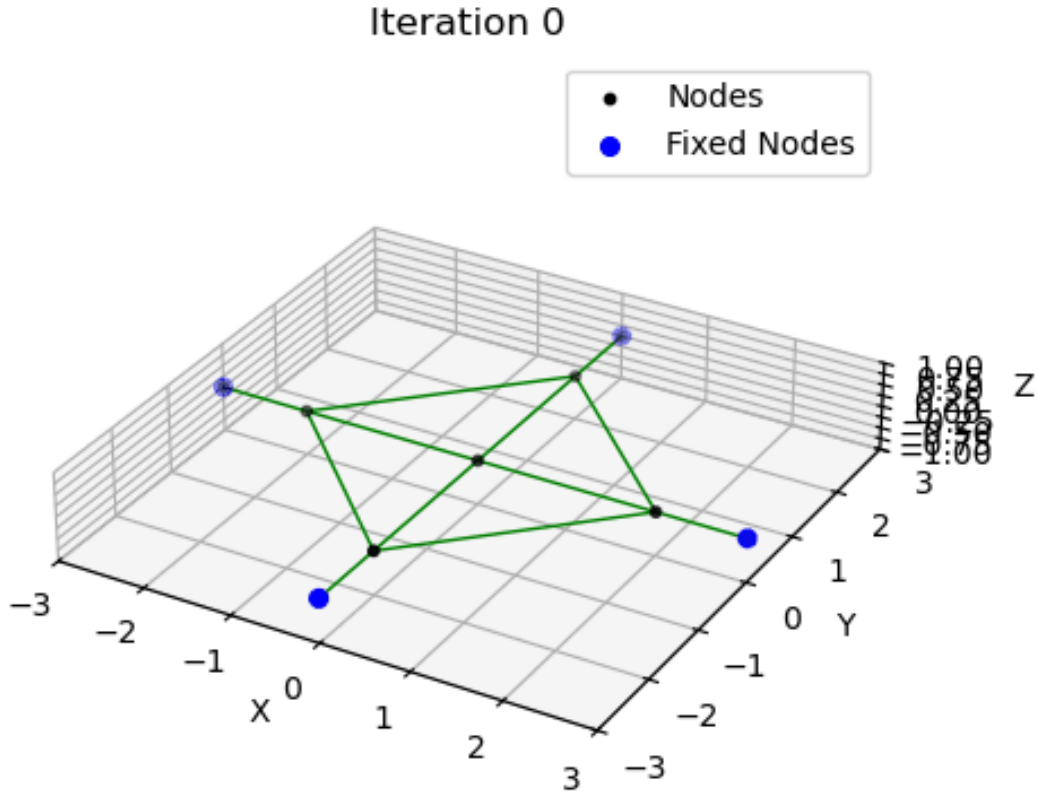
This changes the *relative* force densities.

Key idea:
The structure adapts its geometry based on relative q values — members with higher q tend to become shorter and stiffer in the final form.

Results (Start -> Final)



Animation (Only Visible in Jupyter Notebook)

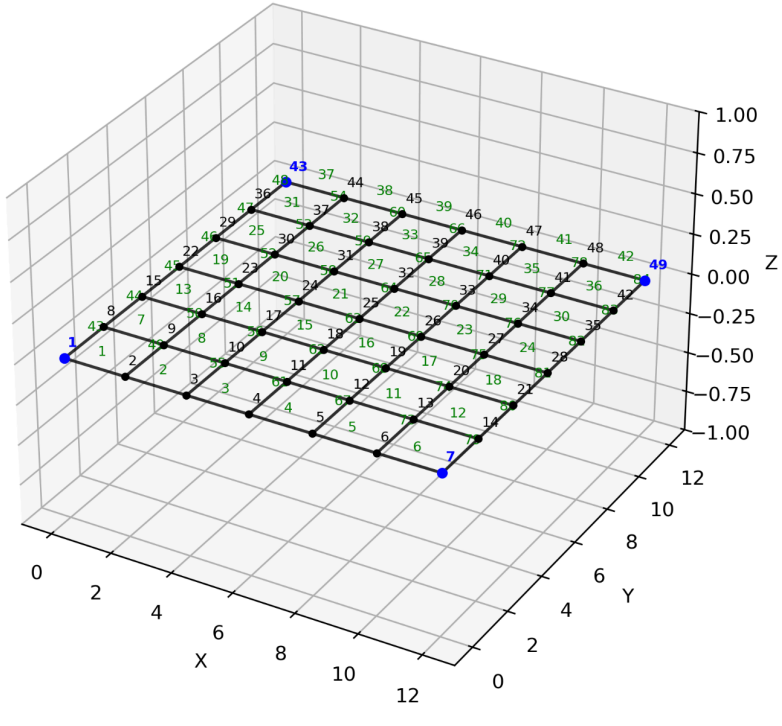


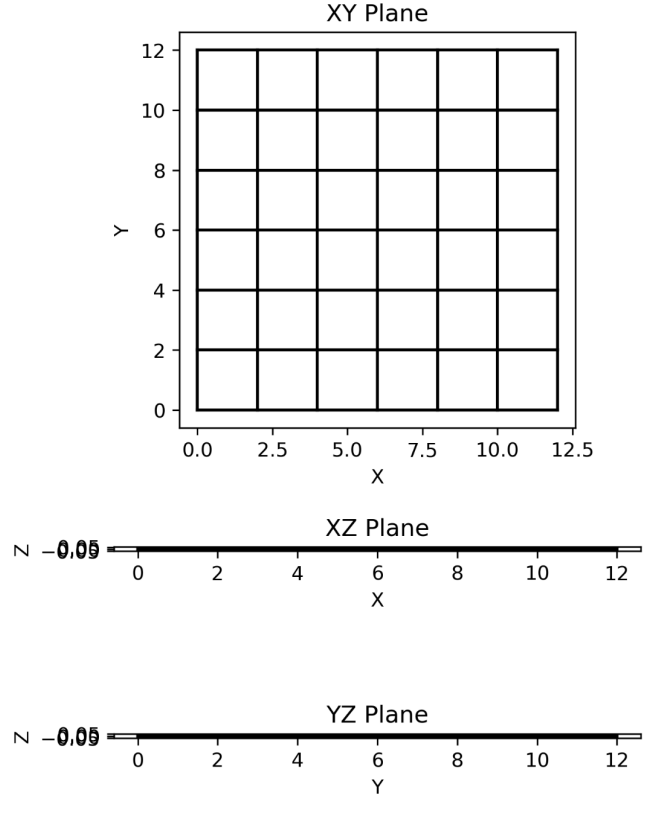
Option 4 — Other Force Density Pattern

Try assigning your own force density pattern. What sort of net structure do you get?

Part 4 - More Complex 2D Net

Description of Structure

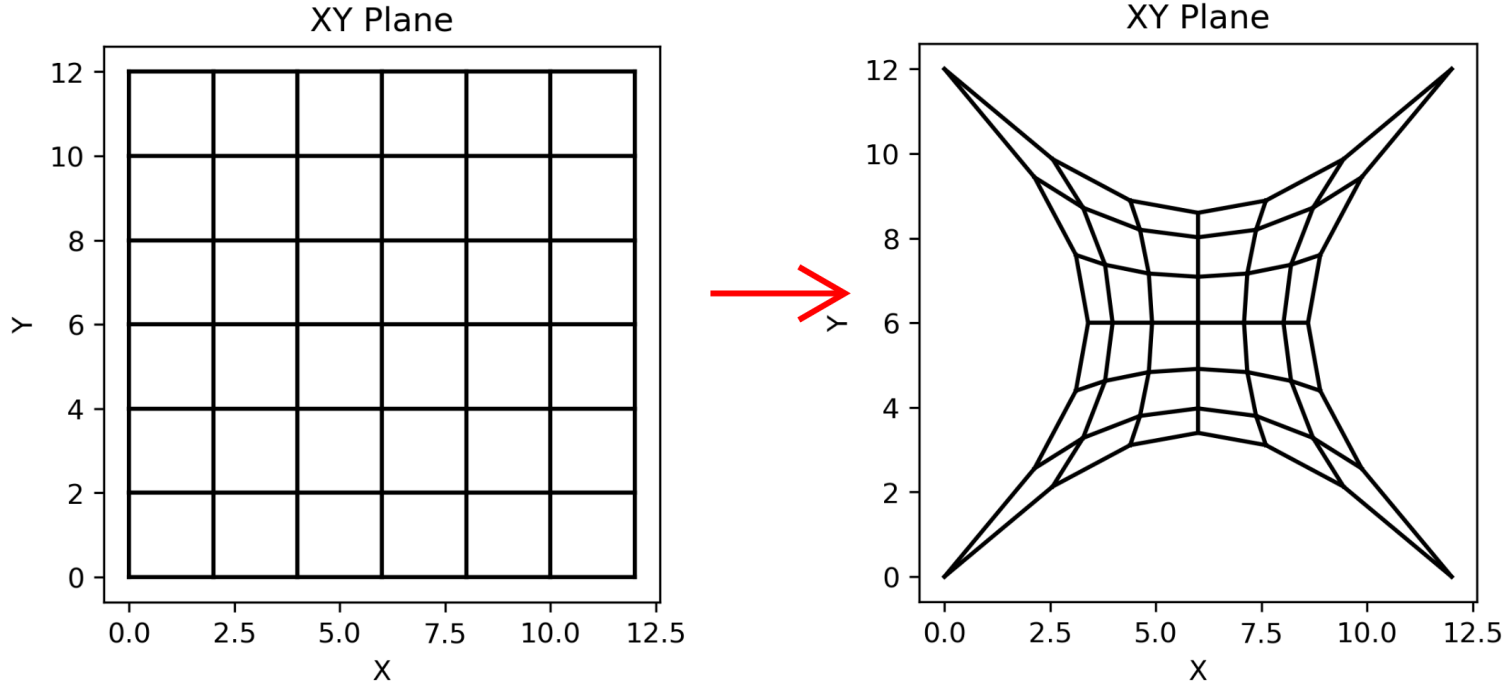




Option 1 — Uniform Force Density

Assume the same force density for every member: $q_i = 1$

Results (Start -> Final)



Option 2 — Edge-Dominated Force Density

Assign larger force densities to the edge members than to the interior members:

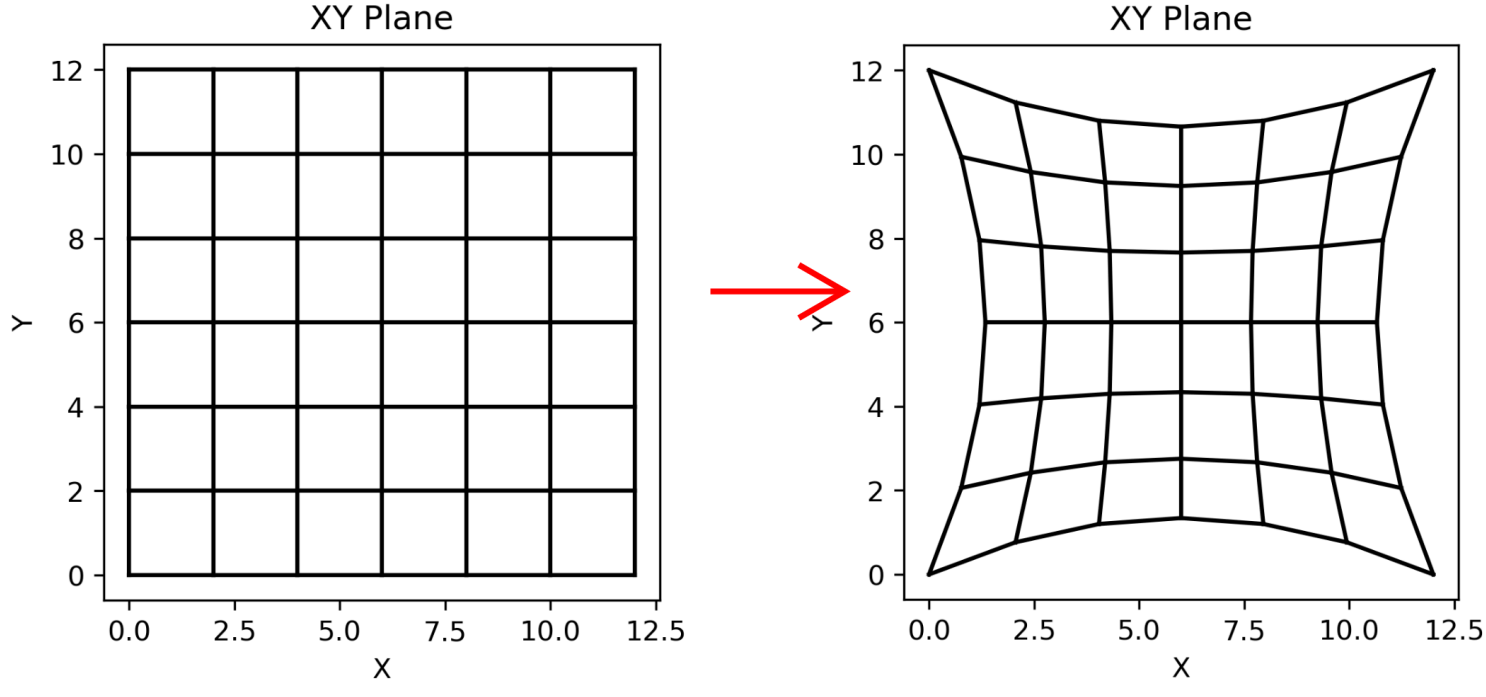
$$q_{\text{edge}} = 5\, q_{\text{interior}}$$

So the ratio is

$$q_{\text{edge}} : q_{\text{interior}} = 5 : 1$$

Remember that only the relative values matter, what do you think making the edges stiffer than the interior will do?

Results (Start -> Final)



Option 3 — Interior-Dominated Force Density

Assign larger force densities to the edge members than to the interior members:

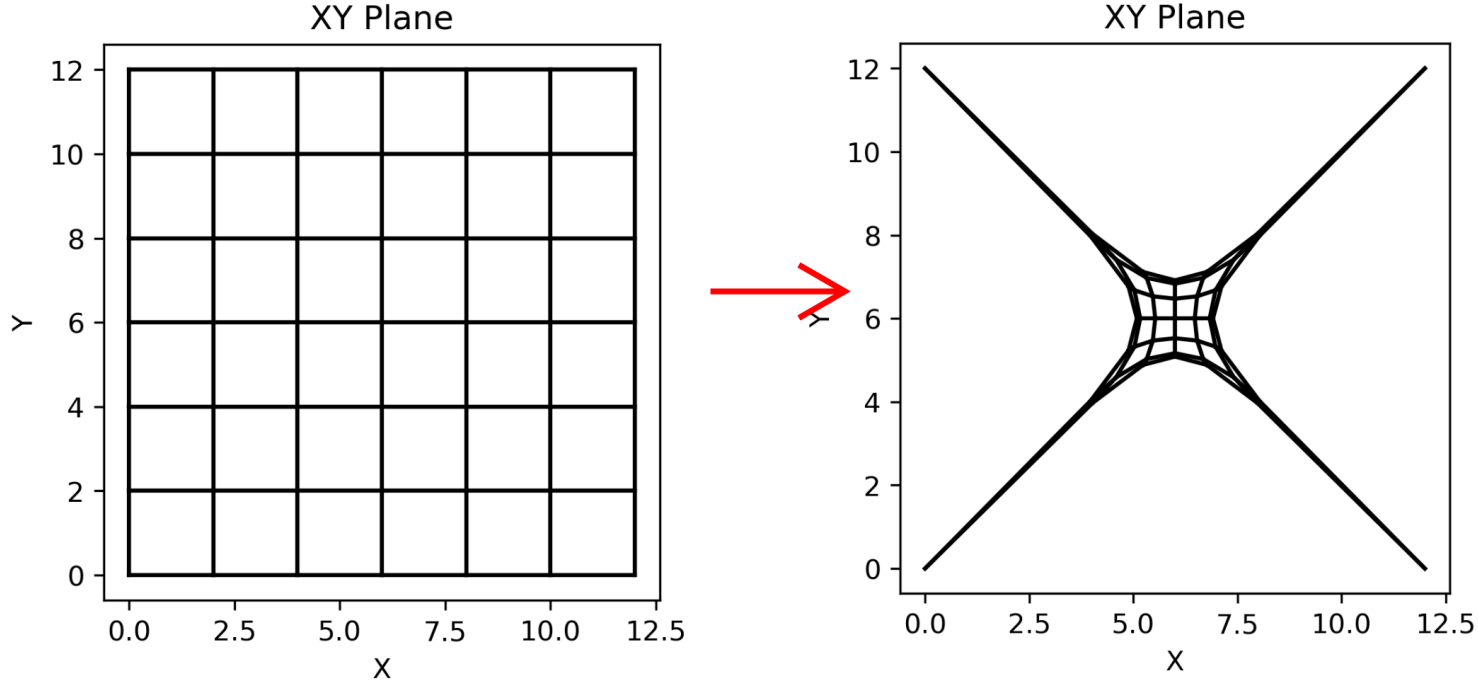
$$q_{\text{interior}} = 5\, q_{\text{edge}}$$

So the ratio is

$$q_{\text{interior}} : q_{\text{edge}} = 5 : 1$$

Remember that only the relative values matter, what do you think making the interior stiffer than the edge will do?

Results (Start -> Final)



Part 5 - Extend Force Density to 3D

Extension to 3D

For 3D form finding, we solve for the out-of-plane coordinates:

$$\mathbf{z} = \mathbf{D}^{-1}\left(\mathbf{p}_z - \mathbf{D}_f \mathbf{z}_f\right)$$

This is directly analogous to the x and y equations.

Key idea:
The same matrix $\mathbf{D}$ and $\mathbf{D_f}$ are used — we simply solve an additional system for the z-direction.

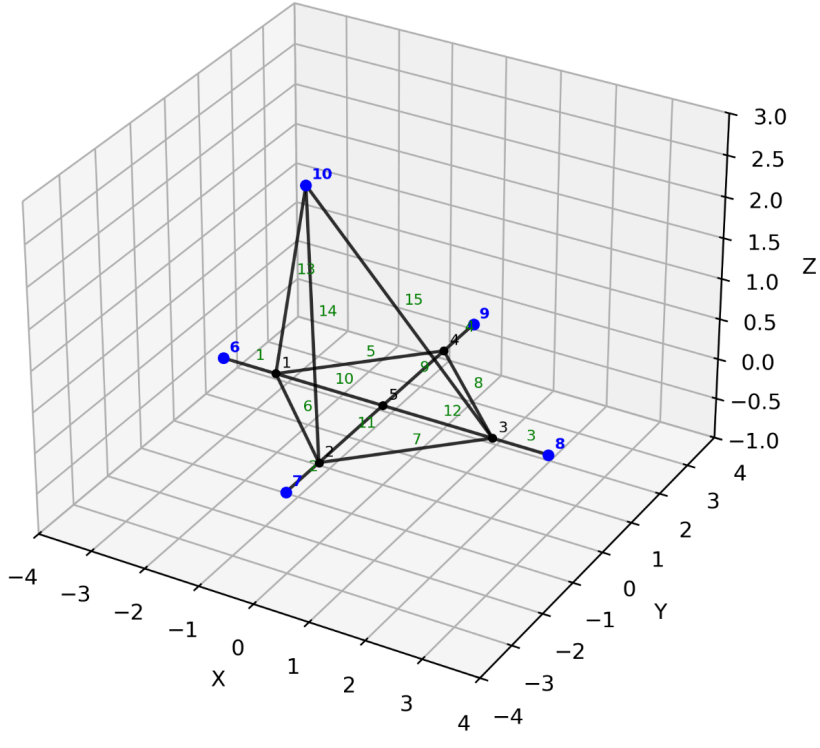
Option 1 — Schek Structure with an Extra Node

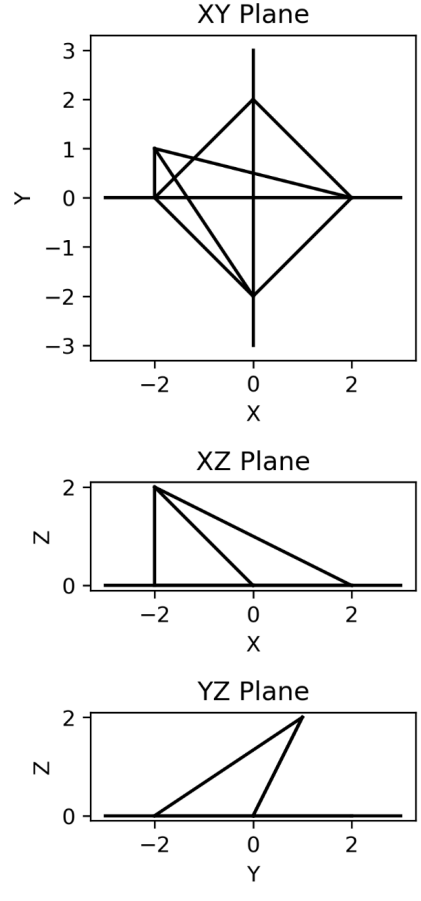
Add one extra node out of plane in the z-direction, connected to the structure by three additional members.

$$\mathbf{q} = [1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1]^T$$

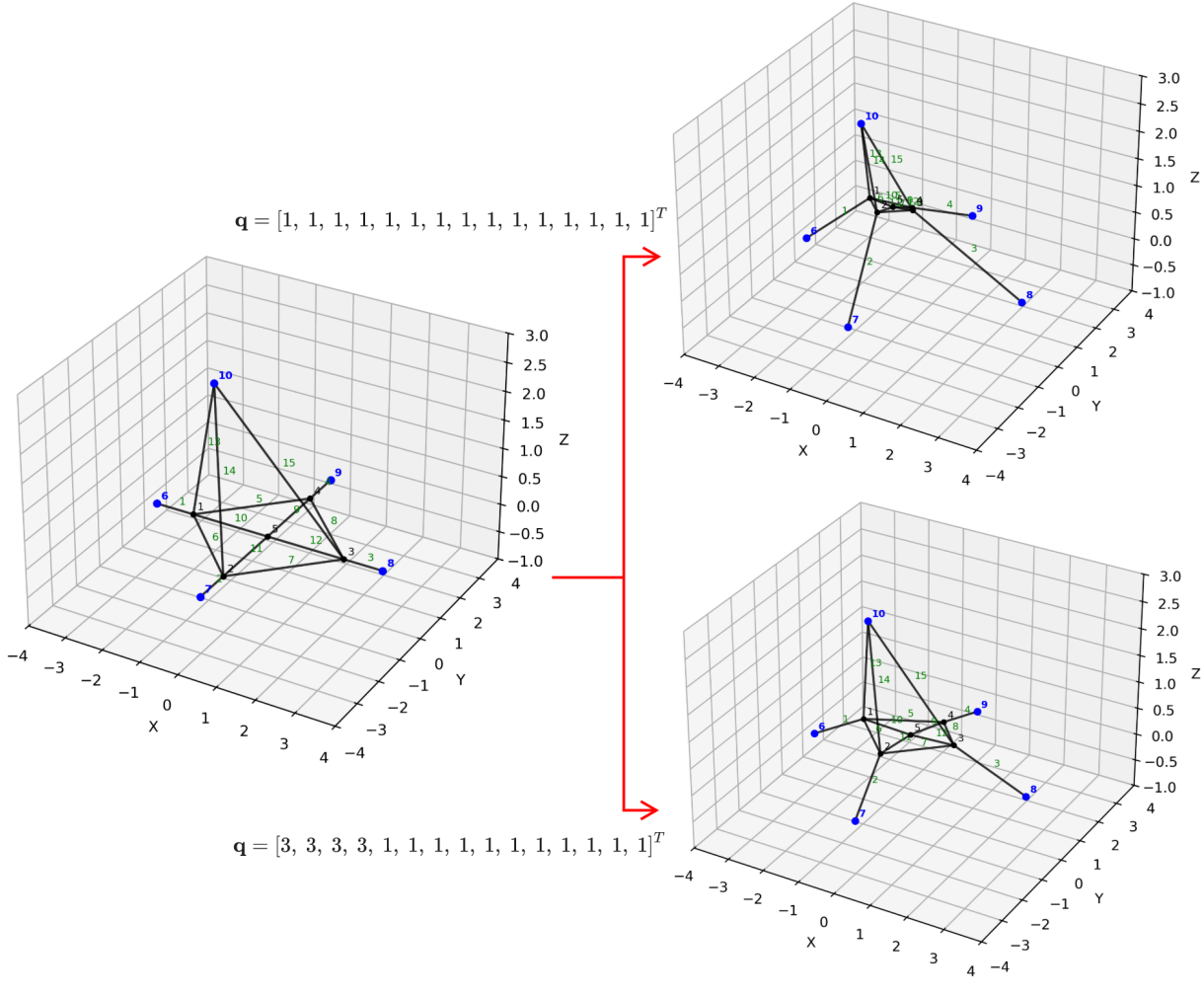
$$\mathbf{q} = [3,\ 3,\ 3,\ 3,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1,\ 1]^T$$

Description of Structure





Results (Start -> Final)



Option 2 — Veenendaal Validation Structure

A hyperbolic paraboloid cable-net with fixed boundaries is used as a validation example to compare all three form-finding methods:

- stiffness matrix method
- dynamic relaxation (with damping)
- force density method

The internal 3×3 mesh has 27 total DOFs.

All analyses start from an initially flat configuration.

Assumptions: $Q = 1$

Post-processing: Forces are normalized for comparison:

$$f_{\text{norm}} = \frac{f}{F_{\text{avg}}}$$

3. Minimal surface

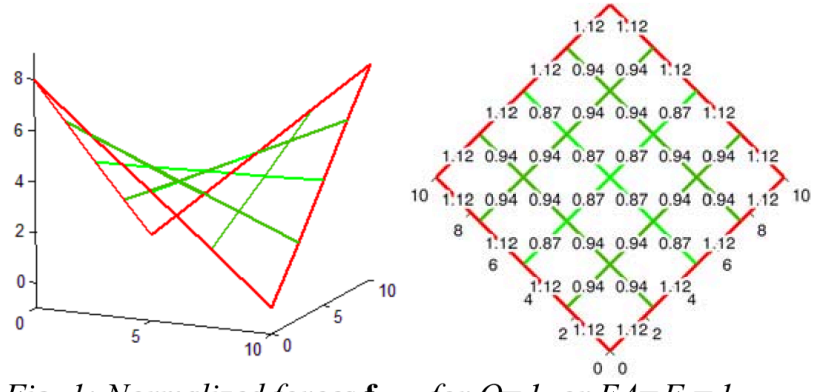
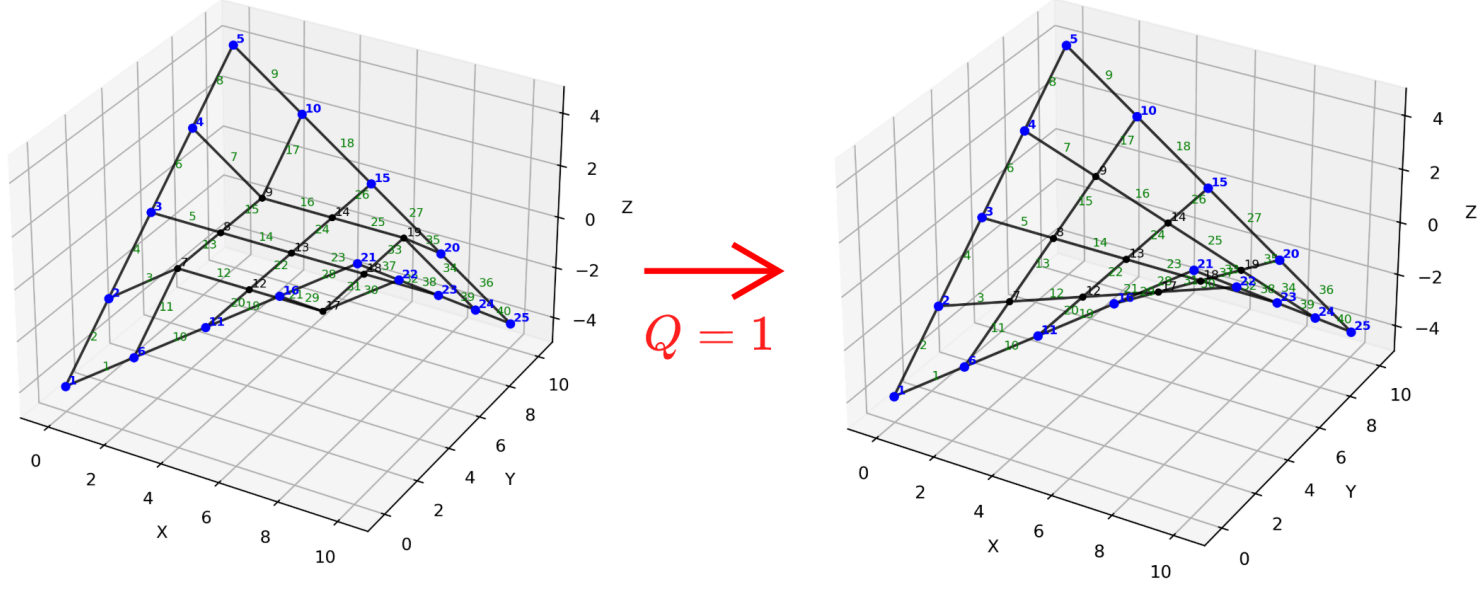


Fig. 1: Normalized forces $\mathbf{f}_{norm}$ *for* $Q=1$, *or* $EA=F_o=1$

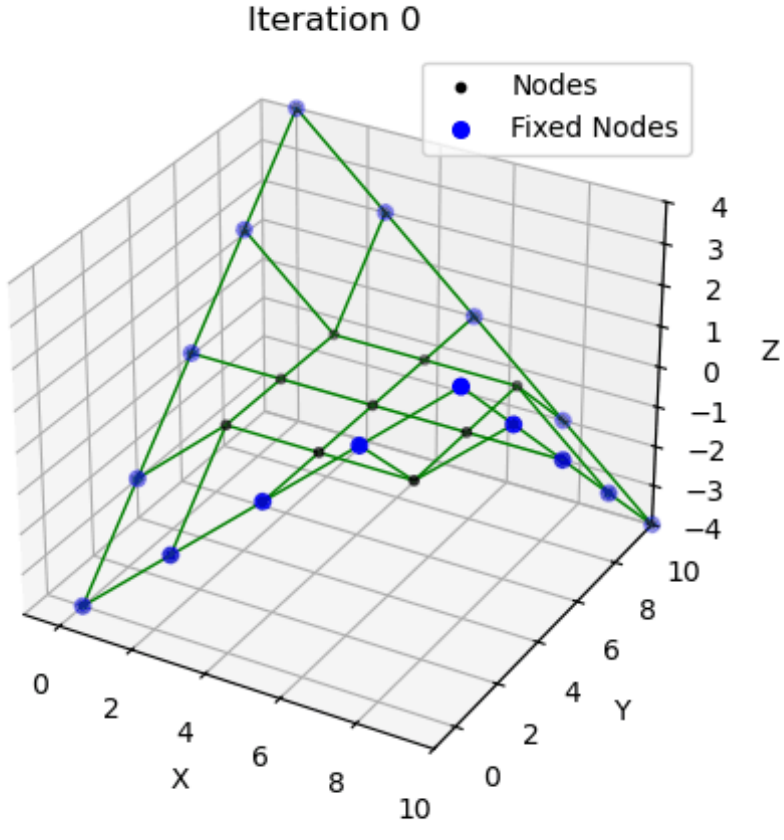
As an example of the framework presented here, the stiffness matrix, dynamic relaxation with viscous and kinetic damping [9] and force density methods are compared by form finding a hyperbolic paraboloid cable-net (Figure 1) with fixed boundaries. Three mesh sizes are computed, with an increasing number of degrees of freedom (3 per node): n=3x3, 27 dof's (Figure 1), n=7x7, 147 dof's and n=15x15, 675 dof's. The form finding starts from an initially flat net (Table 2).

As a first trial, trivial input parameters are used: Q=1, EA=F_0=1 and all unstressed lengths L_0 are equal. The solution is considered to have converged when the squared sum of all residual forces changes by less than 1% between iterations, or when the residual forces are 0. The forces in the results are normalized to their average for comparison ($\mathbf{f}_{norm}=\mathbf{f}/F_{avg}$).

Results (Start -> Final)



Animation (Only Visible in Jupyter Notebook)



Option 3 — Pauletti Validation Structure

Validation example from:

Pauletti & Fernandes (2020) — *An outline of the natural force density method and its extension to quadrilateral elements*
(see L13 folder for the paper)

A dense cable net with pinned supports at opposite corner nodes.
The geometry is extended to 3D by offsetting the corners in the z-direction.

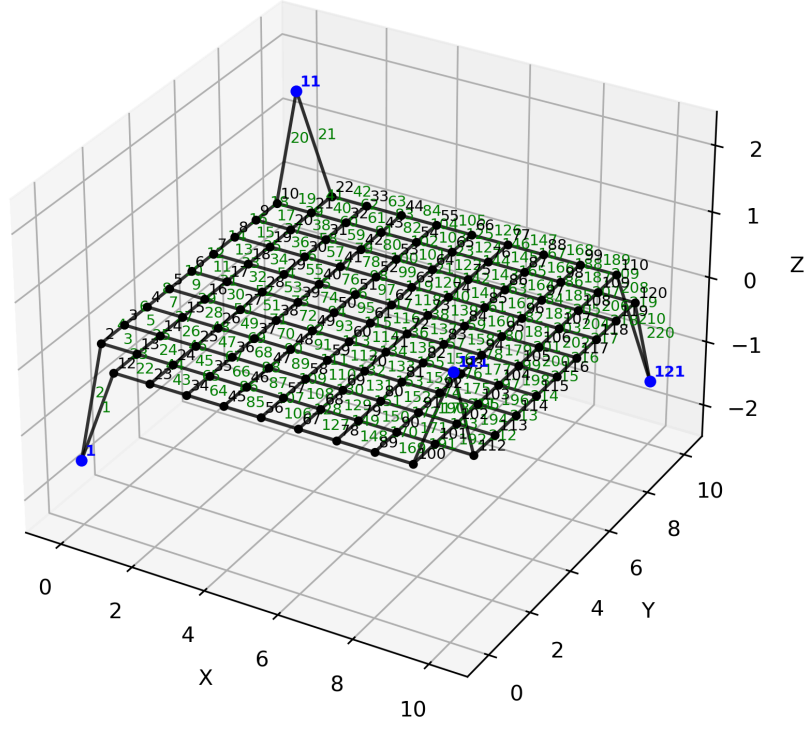
This setup is similar to the 2D "Complex Net", but introduces out-of-plane form finding.

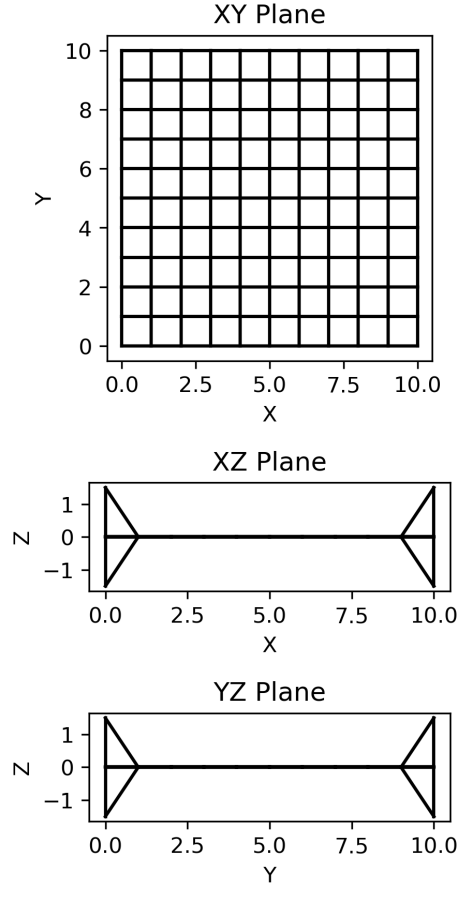
Study parameter:

- Ratio of outer to inner force densities:

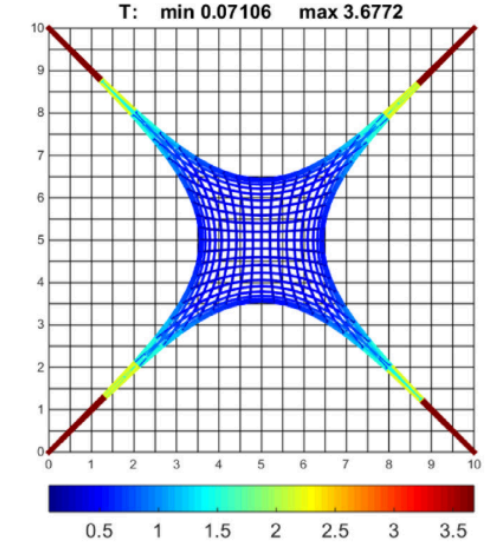
$$q_{\text{outer}} : q_{\text{inner}} = 1:1,\ 5:1,\ 30:1$$

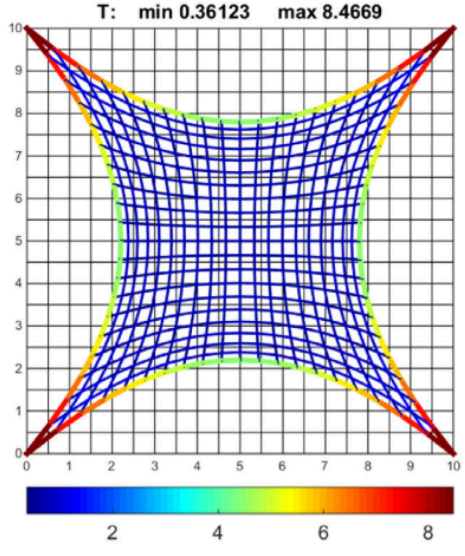
Description of Structure

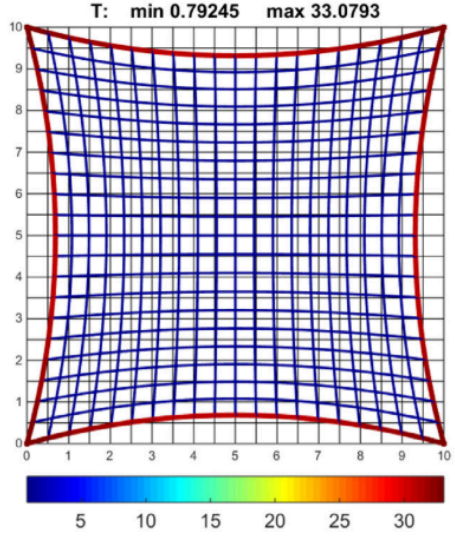


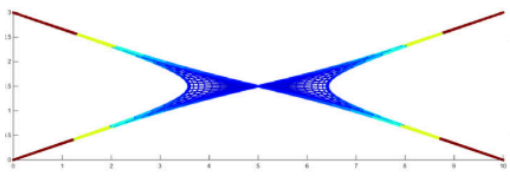


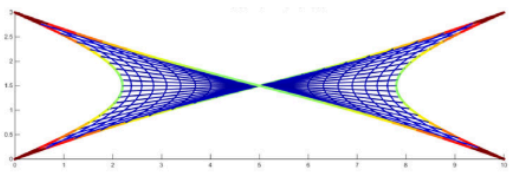
Results from Pauletti Paper

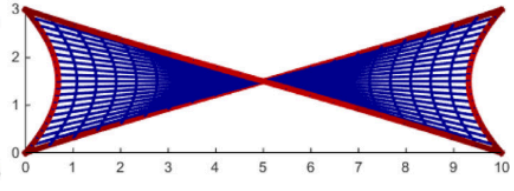






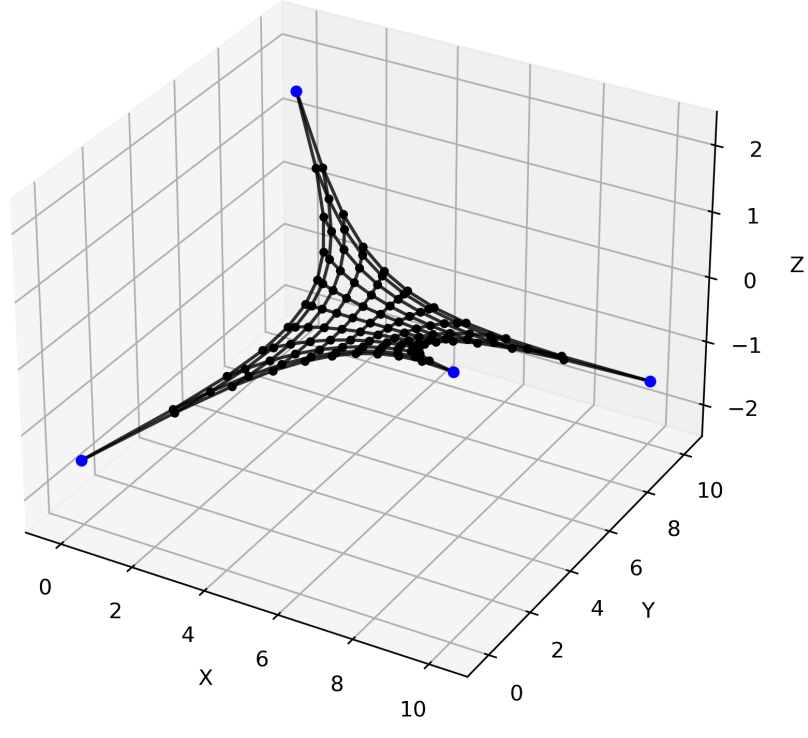


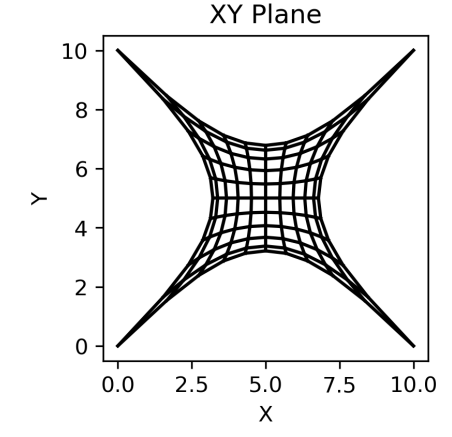


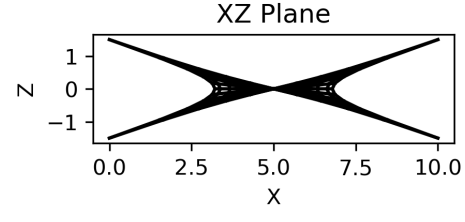


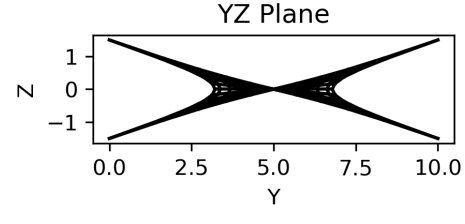
$N_b = 1; N_i = 1$ $N_b = 5; N_i = 1$ $N_b = 30; N_i = 1$

Result from 1:1 Case

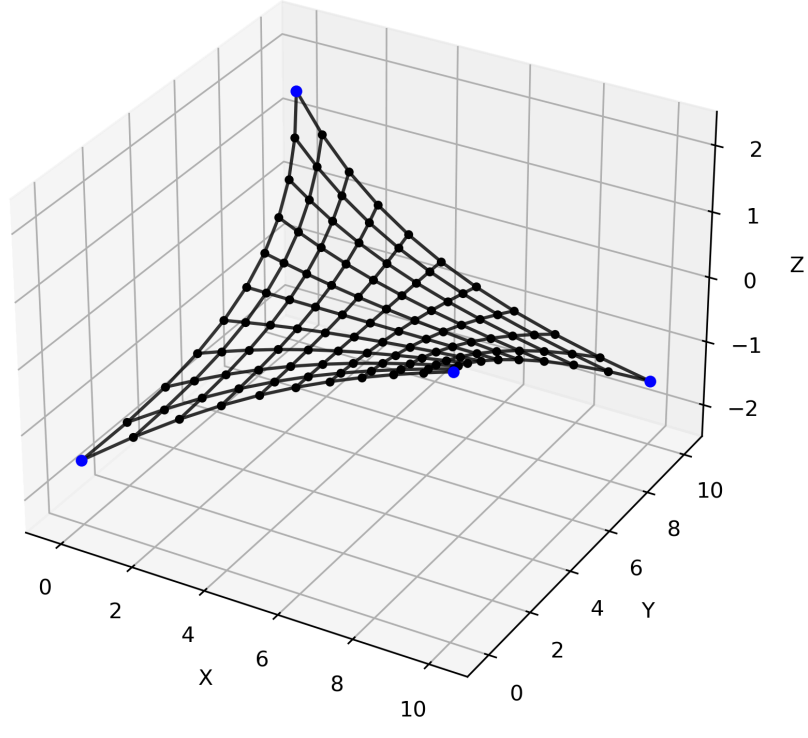


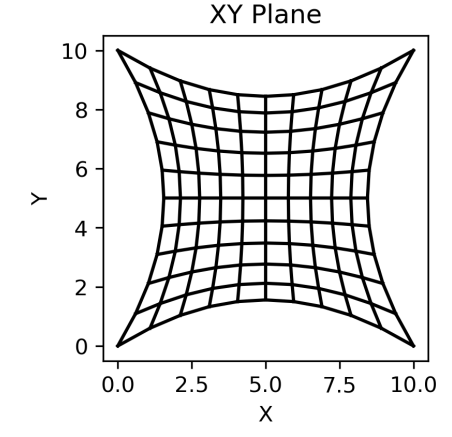


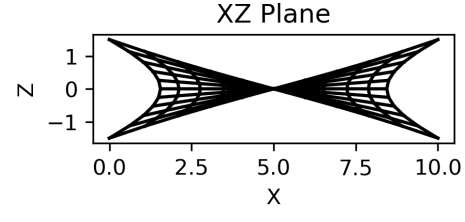


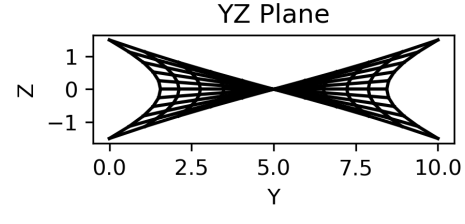


Result from 5:1 Case

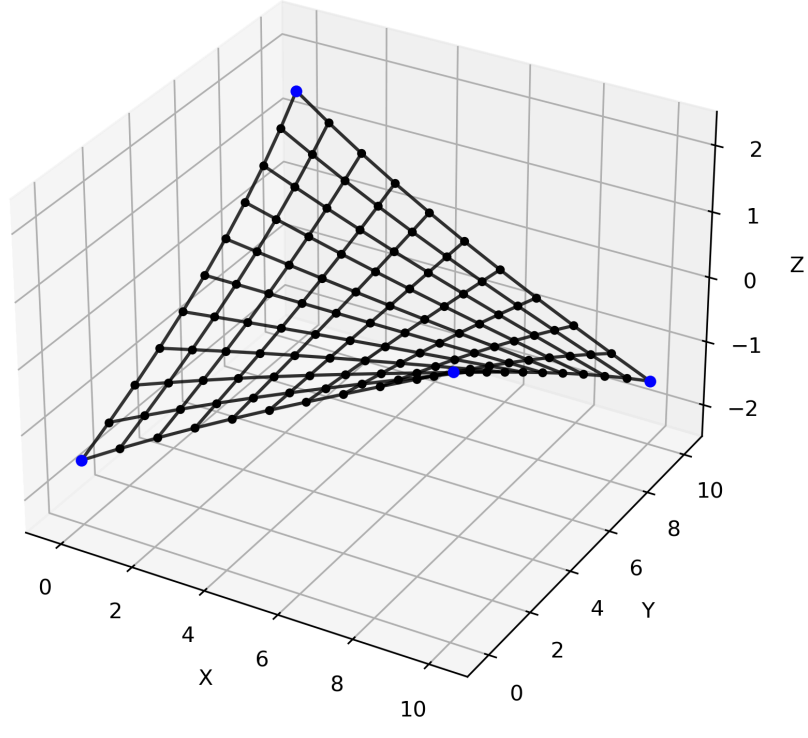


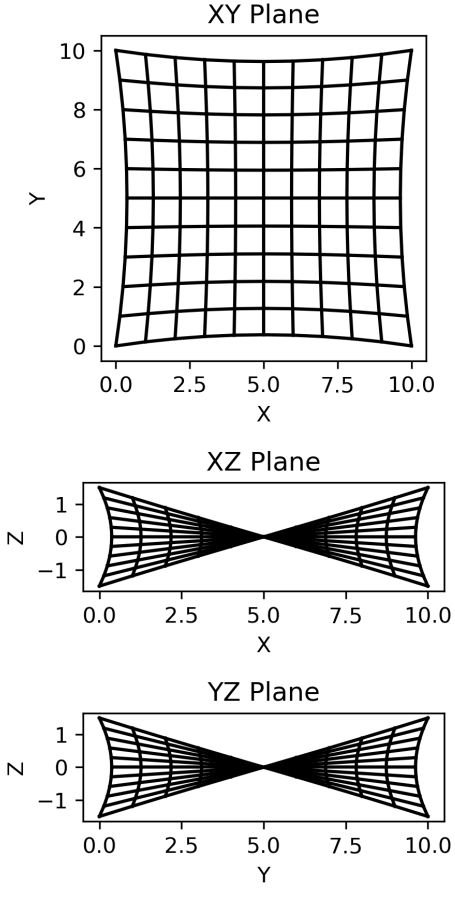






Result from 30:1 Case





Part 6 - Adding Applied Loads

CEE6501 Point Load Example

So far, we have only considered form-finding problems in which the internal force state is prescribed through the force density values. You can think of this as a specified prestress or pretension in the network.

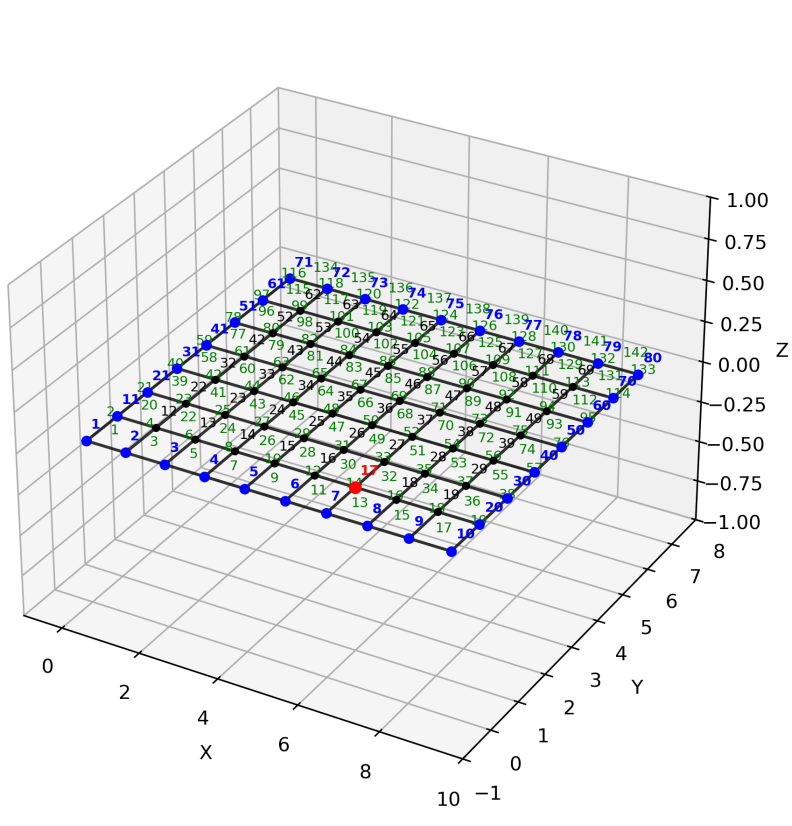
We can also include external applied loads, so the structure must now find a geometry that satisfies equilibrium under both:

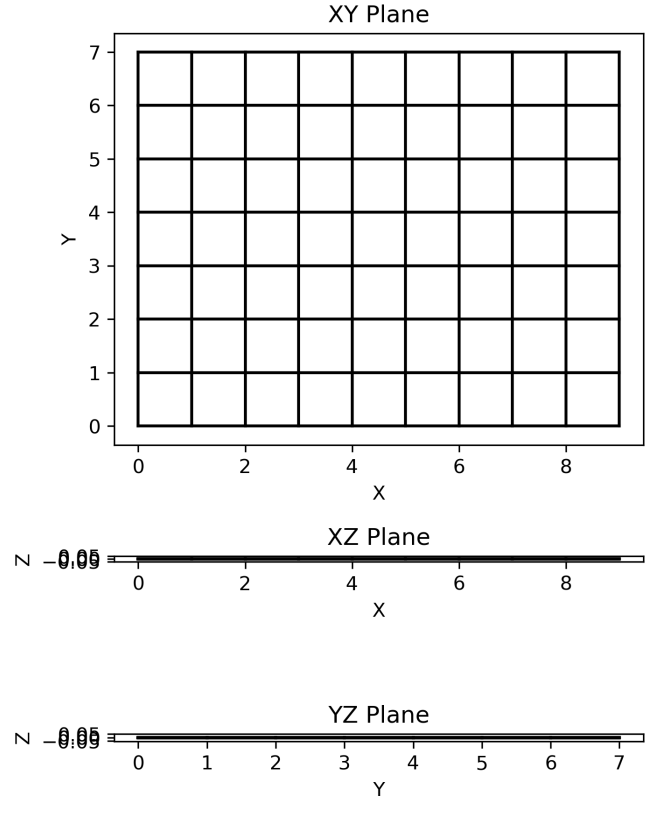
- the prescribed internal force density state, and
- the external nodal loads

Question:
What shape does the net take under these combined conditions?

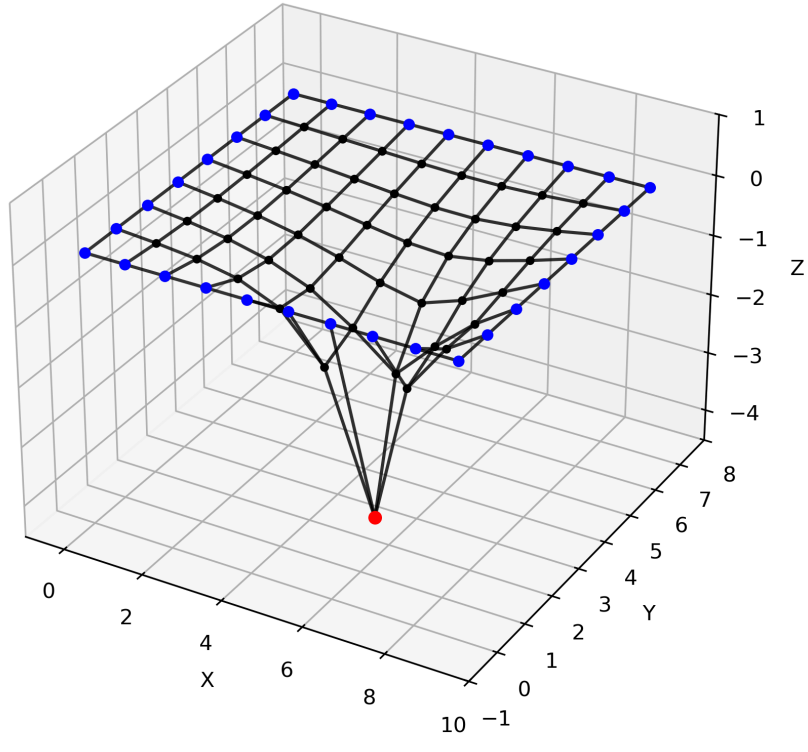
In this example, we prescribe a uniform force density, $q_i = 1$, in all members, and also apply downward point loads at selected nodes.

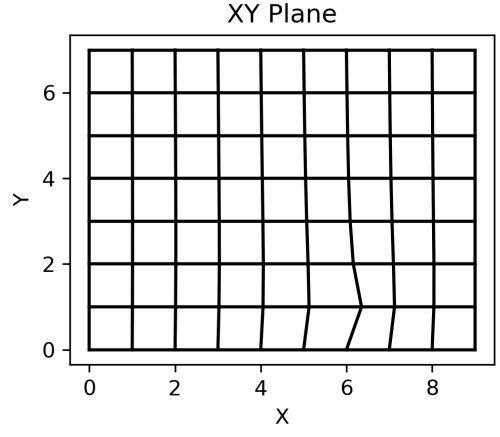
Single Point Load (2.0, 0.0, -10.0) at node 17

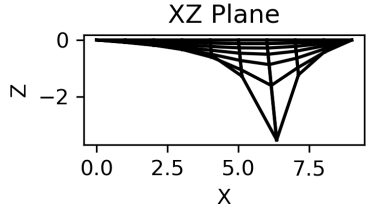


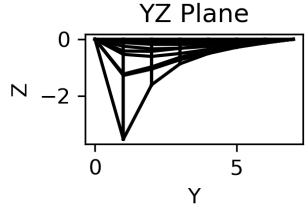


Result from Single Point Case

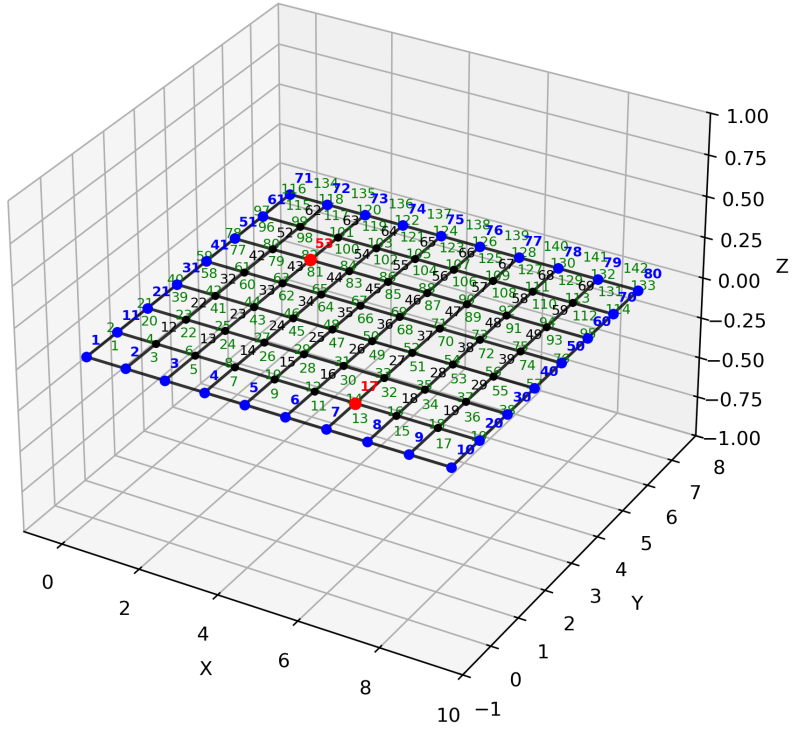


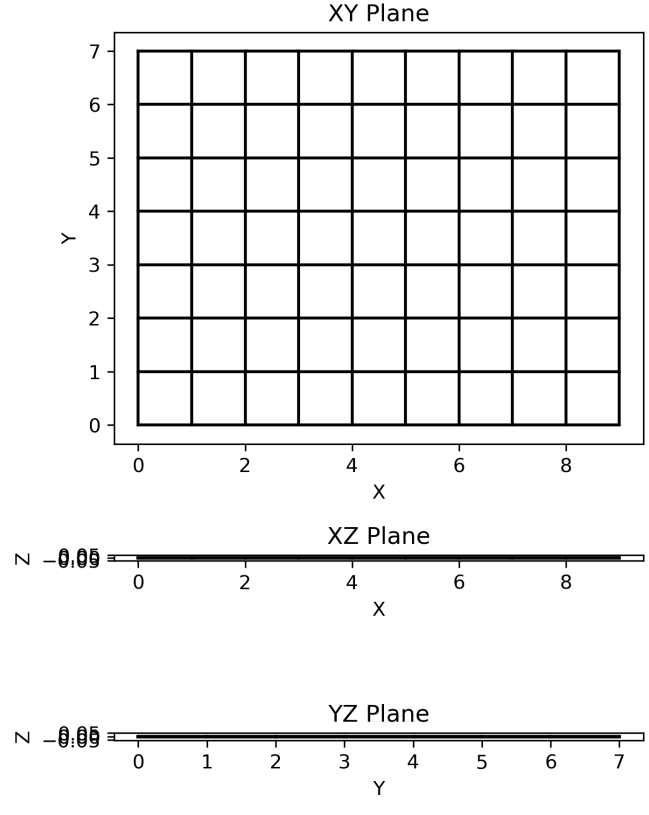




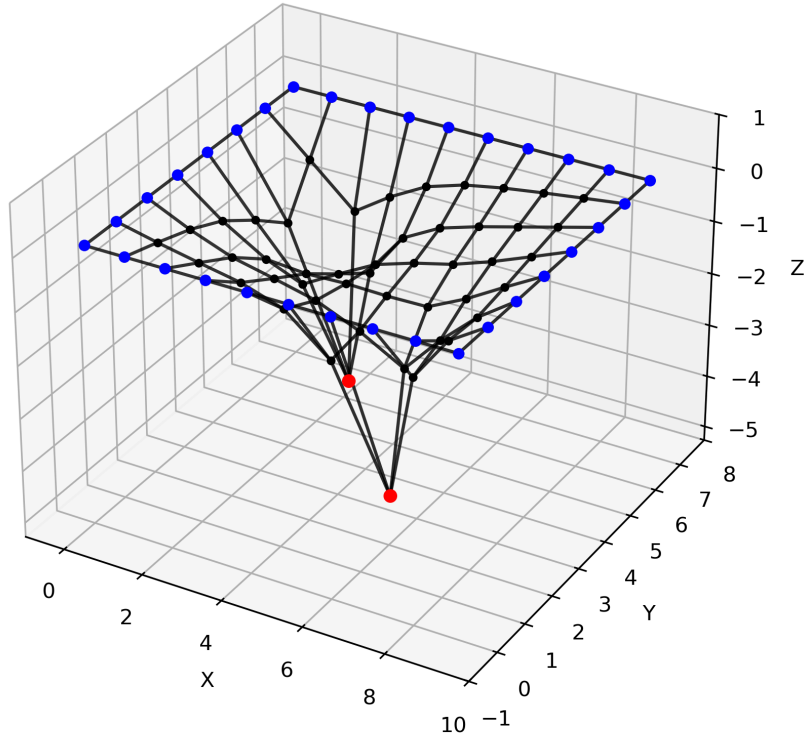


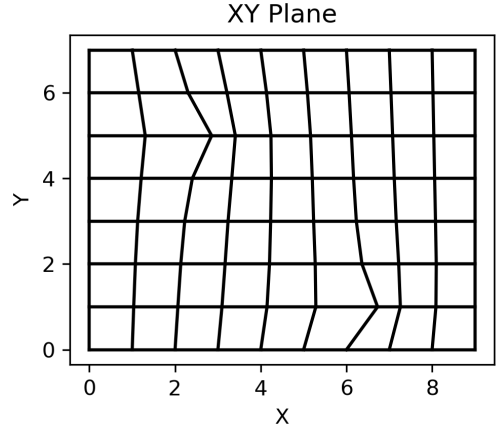
Two Point Loads (2.0, 0.0, -10.0) at node 17 and 53

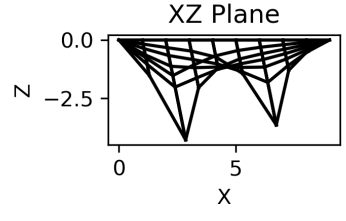


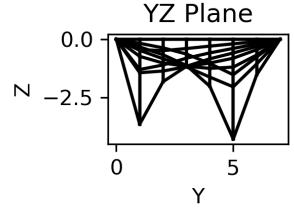


Result from Double Point Case

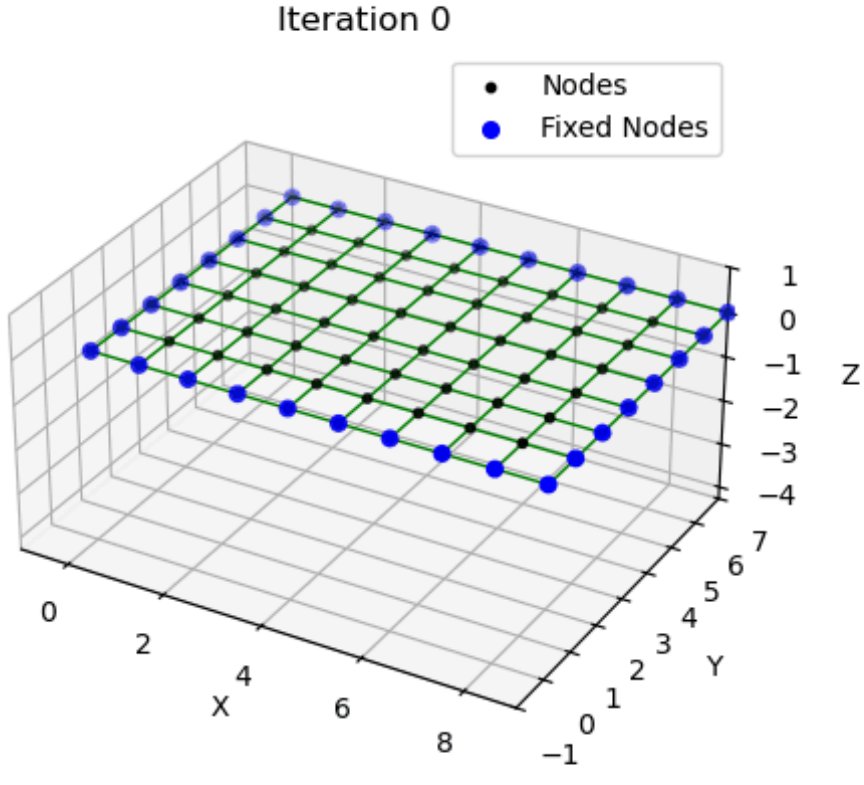








Animation (Only Visible in Jupyter Notebook)



Wrap-Up

In this lecture, we:

-

Main takeaway: Once the connectivity, supports, loads, and force densities are prescribed, the force density method finds the equilibrium geometry through a linear solve.